

Aula 3 curso python - Coleções e sequencias

1.Estruturas de dados: Métodos universais

Quando precisamos agrupar vários valores em uma única variável, usamos estruturas de dados. Que são sequências de objetos(variáveis) alocados dinamicamente ja nativo em python, o que as tornam muito mais versáteis que vetores normais em c.

Ou seja, qualquer “vetor” em python por ser alocado dinamicamente nos permite aumentar ou diminuir o vetor durante a execução, sem a necessidade de alocar.

Em python existem quatro tipos de Vetores que podem ser utilizados

- listas [**list**]
- Tuplas (**tuple**)
- Conjuntos **{set}**
- Dicionários **{dict}**

Funções Nativas Usadas com Sequências ordenadas

Estas são funções gerais do Python que são muito úteis com listas.

- `len()`: Retorna o número de elementos na lista.
 - **Funciona com:** Tuplas, Strings, Conjuntos e Dicionários
- `sum()`: Retorna a soma de todos os elementos (deve conter apenas números).
 - **Strings:** Funciona com strings somando tabela ASCII
 - **Listas, Tuplas e Conjuntos:** Funciona, desde que contenham apenas números.
 - **Dicionários:** Não diretamente. Você pode somar os valores (`sum(meu_dicionario.values())`) ou as chaves se forem numéricas (`sum(meu_dicionario.keys())`).
- `min()`: Retorna o menor elemento da lista.
- `max()`: Retorna o maior elemento da lista.
 - Ambos `min()` e `max()` funcionam com qualquer iterável cujos itens possam ser comparados entre si.
- `sorted()`: Retorna uma nova lista ordenada, sem modificar a original.
 - **Funciona com:** Tuplas, Strings, Conjuntos e Dicionários

indexação negativa

Usar `lista[-1]` é a maneira mais comum em Python para aceder ao **último elemento** de uma lista.

Isso é chamado de indexação negativa. Funciona assim:

- `lista[-1]` acessar o último elemento.
- `lista[-2]` acessa o penúltimo elemento.
- E assim por diante.

A indexação negativa funciona também para:

- **Listas** (**list**): `[1, 2, 3][-1]` retorna 3.
- **Tuplas** (**tuple**): `(1, 2, 3)[-1]` retorna 3.
- **Strings** (**str**): `"abc"[-1]` retorna 'c'.

2. Listas (list)

Sintaxe: Elementos separados por vírgula, dentro de colchetes `[]`.

Características Principais:

- **Mutável:** Você pode alterar, adicionar e remover elementos depois que a lista é criada.
- **Ordenada:** Os elementos mantêm a ordem em que foram adicionados.
- **Indexada:** Você pode acessar qualquer elemento pelo seu índice numérico (começando em 0).
- **Permite Duplicados:** Uma lista pode conter elementos repetidos sem problemas

```
1 # Criando uma lista de compras
2 compras = ["maçã", "banana", "leite", "pão"]
3
4 # Acessando itens pelo índice (começa em 0)
5 print(f"O primeiro item é: {compras[0]}")
6
7 # Adicionando um item ao final
8 compras.append("queijo")
9 print(f"Lista com novo item: {compras}")
10
11 # Removendo o último item
12 compras.pop()
13 # ou pop(0) para remover o primeiro item
14 print(f"Lista após remover item: {compras}")
```

2.1. Usando as métodos de Listas:

.append(elemento): Adiciona um único elemento ao **final** da lista.

```
1 frutas = ["maçã", "banana"]
2 frutas.append("laranja")
3 # frutas agora é ['maçã', 'banana', 'laranja']
```

.extend(iterável): Adiciona todos os elementos de um iterável (como outra lista) ao final da lista.

```
1 frutas = ["maçã", "banana"]
2 outras_frutas = ["uva", "pêra"]
3 frutas.extend(outras_frutas)
4 # frutas agora é ['maçã', 'banana', 'uva', 'pêra']
```

.insert(índice, elemento): Insere um elemento numa posição específica.

```
1 frutas = ["maçã", "banana"]
2 frutas.insert(1, "laranja")
  # Insere na posição do índice 1
3 # frutas agora é ['maçã', 'laranja', 'banana']
```

.remove(elemento): Remove a **primeira ocorrência** do elemento especificado. Gera um erro se o elemento não existir.

```
1 frutas = ["maçã", "banana", "maçã"]
2 frutas.remove("maçã")
3 # frutas agora é ['banana', 'maçã']
```

.pop(índice): Remove e retorna o elemento de um índice específico. Se nenhum índice for fornecido, remove e retorna o **último** elemento.

```
1 frutas = ["maçã", "banana", "laranja"]
2 fruta_removida = frutas.pop(1) # Remove 'banana'
3 # fruta_removida é 'banana'
4 # frutas agora é ['maçã', 'laranja']
```

.clear(): Remove todos os elementos da lista, deixando-a vazia.

```
1 frutas = ["maçã", "banana"]
2 frutas.clear()
3 # frutas agora é []
4
```

.sort(): Ordena os itens da lista (in-place). Por padrão, em ordem crescente.

```
1 numeros = [3, 1, 4, 2]
2 numeros.sort()
3 # numeros agora é [1, 2, 3, 4]
```

.reverse(): Inverte a ordem dos elementos da lista (in-place).

```
1 frutas = ["maçã", "banana", "laranja"]
2 frutas.reverse()
3 # frutas agora é ['laranja', 'banana', 'maçã']
```

.count(elemento): Retorna o número de vezes que um elemento aparece na lista.

```
1 numeros = [1, 2, 3, 2, 2]
2 contagem = numeros.count(2)
3 # contagem é 3
```


.index(elemento): Retorna o índice da primeira ocorrência de um elemento. Gera um erro se o elemento não for encontrado.

```
1 frutas = ["maçã", "banana", "laranja"]
2 indice = frutas.index("banana")
3 # indice é 1
```

.copy(): Retorna uma cópia da lista.

```
1 lista_original = [1, 2, 3]
2 lista_copia = lista_original.copy()
3
4 # caso voce atribua diretamente,
5 # a lista que esta recebendo,se torna uma referência a lista original
6 lista_copia = lista_original
7 lista_original[0] = 10
8 # lista_copia sera [10, 2, 3, 2, 2]
```

3. Tuplas (tuple)

Uma tupla é muito parecida com uma lista, mas com uma diferença crucial: ela é **imutável**. Uma vez criada, uma tupla não pode ser alterada.

- **Sintaxe:** Elementos separados por vírgula, dentro de parênteses `()`.
- **Características Principais:**
 - **Imutável:** Não se pode adicionar, remover ou alterar elementos.
 - **Ordenada:** Assim como as listas, a ordem dos elementos é preservada.
 - **Indexada:** Os elementos podem ser acessados por seu índice numérico.
 - **Permite Duplicados:** Também pode conter elementos repetidos.

```
1 # Criando uma tupla para coordenadas RGB
2 cor_primaria = (255, 0, 0) # Vermelho
3
4 # Acessando um item
5 print(cor_primaria[0]) # Saída: 255
6
7 # Tentando modificar (isto causará um erro!)
8 # cor_primaria[0] = 100 # TypeError: 'tuple' object
  # does not support item assignment
```

3.1. A Imutabilidade na Prática: Por que não há funções de modificação?

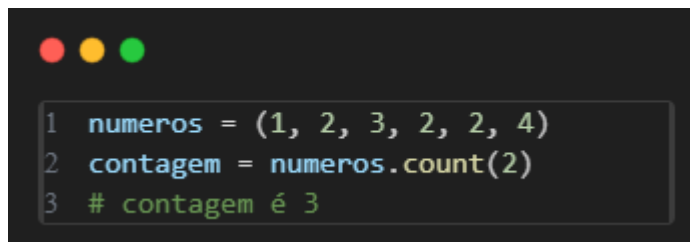
A ausência de métodos que modificam a tupla (como `.append()`, `.remove()`, `.pop()`, `.sort()`, etc.) não é uma limitação, mas sim a principal **característica** das tuplas.

- **Segurança de Dados:** Garante que os dados permaneçam constantes ao longo do programa.
- **Performance:** Tuplas são ligeiramente mais rápidas e consomem menos memória que as listas.
- **Uso como Chaves de Dicionário:** A imutabilidade permite que tuplas sejam usadas como chaves em dicionários, algo que as listas não podem fazer.

3.2. Métodos de Tuplas

Por serem imutáveis, as tuplas possuem apenas dois métodos, e ambos servem para obter informações, sem alterar a tupla original.

`.count(elemento)`: Retorna o número de vezes que um elemento aparece na tupla.

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The code is written in Python and demonstrates the use of the `count` method on a tuple.

```
1 numeros = (1, 2, 3, 2, 2, 4)
2 contagem = numeros.count(2)
3 # contagem é 3
```

.index(elemento): Retorna o índice da primeira ocorrência de um elemento. Gera um erro (**ValueError**) se o elemento não for encontrado.

```
1 frutas = ("maçã", "banana", "laranja", "banana")
2 indice = frutas.index("banana")
3 # indice é 1
```

3.3. Como "Modificar" uma Tupla

Se você realmente precisa modificar os dados de uma tupla, o padrão correto é criar uma nova tupla. Mas podemos fazer uma conversão temporária para uma lista.

```
1 coordenadas = (10, 20)
2
3 # 1. Converter a tupla para uma lista
4 lista_temporaria = list(coordenadas)
5
6 # 2. Modificar a lista
7 lista_temporaria.append(30)
8
9 # 3. Converter a lista de volta para uma tupla
10 novas_coordenadas = tuple(lista_temporaria)
11
12 print(f"Novas coordenadas: {novas_coordenadas}")
13 # Saída: Novas coordenadas: (10, 20, 30)
```

4. Conjuntos (set)

Um conjunto é uma coleção de itens **únicos** e **não ordenada**. Sua principal utilidade é verificar rapidamente se um item pertence à coleção e remover elementos duplicados.

- **Sintaxe:** Elementos separados por vírgula, dentro de chaves `{}` (sem pares chave-valor). Para criar um conjunto vazio, use `set()`.
- **Características Principais:**
 - **Mutável:** Você pode adicionar e remover elementos.
 - **Não Ordenado:** Não há garantia sobre a ordem dos elementos.
 - **Não Indexado:** Você não pode acessar elementos por um índice.
 - **Não Permite Duplicados:** Se você adicionar um item que já existe, nada acontece.

```
1 # Criando um conjunto de tags
2 tags = {"python", "dados", "web", "python"}
3
4 print(tags) # Saída: {'web', 'python', 'dados'}
5 #(a ordem pode variar e "python" aparece só uma vez)
6
7 # Verificando se um item existe (muito rápido!)
8 print("python" in tags) # Saída: True
9
10 # Adicionando um novo item
11 tags.add("ia")
12 print(tags) # Saída: {'web', 'python', 'ia', 'dados'}
13
14 # Removendo duplicados de uma lista
15 numeros = [1, 2, 3, 2, 4, 1, 5]
16 numeros_unicos = set(numeros)
17 print(numeros_unicos) # Saída: {1, 2, 3, 4, 5}
```

4.1. Funções e métodos de Conjuntos:

Set(): Para criar um conjunto a partir de qualquer objeto iterável (como uma lista ou tupla), removendo duplicatas no processo.

- **Importante:** Para criar um conjunto vazio, você **deve** usar `set()`, pois `{}` cria um dicionário vazio.

```
1 numeros_lista = [1, 2, 3, 2, 4, 1, 5]
2 numeros_unicos = set(numeros_lista)
3 print(numeros_unicos)
4 # Saída: {1, 2, 3, 4, 5}
```

.add(elemento): Adiciona um único elemento ao conjunto. Se o elemento já existir, nada acontece.

```
1 linguagens = {"python", "java"}
2 linguagens.add("javascript")
3 linguagens.add("python")
4 # Não faz nada, pois "python" já existe
5 print(linguagens)
6 # Saída: {'java', 'javascript', 'python'}
```

.update(iterável): Adiciona todos os itens de um iterável (como outra lista ou conjunto) ao conjunto.

- Caso utilizado em um string, cada caracter se torna um elemento

```
1 linguagens = {"python", "java"}
2 linguagens.update(["c++", "ruby", "java"])
3 print(linguagens)
# Saída: {'ruby', 'c++', 'java', 'javascript', 'python'}
4
5 alfabeto = set()
6 alfabeto.update("abcde")
7 # Adiciona cada letra como um elemento separado
8 print(alfabeto)
9 # saída: {'a', 'b', 'c', 'd', 'e'}
```

.remove(elemento): Remove um elemento. Gera um erro (**KeyError**) se o elemento não existir.

```
1 linguagens.remove("java")
2 print(linguagens)
# Saída: {'ruby', 'c++', 'javascript', 'python'}
3 # linguagens.remove("c#") # Isto geraria um KeyError
```

`.discard(elemento)`: Remove um elemento, mas **não** gera um erro se o elemento não for encontrado. É mais seguro que `.remove()`

```
1 linguagens.discard("c#") # Nenhuma ação, nenhum erro
```

`.clear()`: Remove todos os elementos, deixando o conjunto vazio.

4.2. Operações Matemáticas de Conjuntos

Usando conjuntos em python é possível usar lógicas matemáticas, as mesmas já vistas em matemática discreta.

Usaremos as mesmas variáveis para os exemplos a seguir

```
1 devs_a = {"Ana", "Caio", "Rafaela", "Daniel"}
2 devs_b = {"Rafaela", "Daniel", "Gustavo", "Fernando"}
```


União (| ou .union()): Retorna um novo conjunto com todos os elementos de ambos os conjuntos, sem duplicatas.

```
1 # Usando o operador |
2 todos_devs = devs_a | devs_b
3 print(f"União: {todos_devs}")
4
5 # Usando o método .union()
6 # todos_devs = devs_a.union(devs_b)
7 # Saída: União: {'Gustavo', 'Daniel', 'Fernando', 'Rafaela', 'Caio', 'Ana'}
```

Interseção (& ou .intersection()): Retorna um novo conjunto contendo apenas os elementos que existem em **ambos** os conjuntos.

```
1 # Usando o operador &
2 devs_comuns = devs_a & devs_b
3 print(f"Interseção: {devs_comuns}")
4
5 # Usando o método .intersection()
6 # devs_comuns = devs_a.intersection(devs_b)
7 # Saída: Interseção: {'Rafaela', 'Daniel'}
```

Diferença (- ou `.difference()`): Retorna os elementos que estão no primeiro conjunto, mas **não** no segundo.

```
1 # Usando o operador -
2 devs_apenas_em_a = devs_a - devs_b
3 print(f"Apenas na equipe A: {devs_apenas_em_a}")
4 # Saída: Apenas na equipe A: {'Ana', 'Bruno'}
5
6 devs_apenas_em_b = devs_b - devs_a
7 print(f"Apenas na equipe B: {devs_apenas_em_b}")
8 # Saída: Apenas na equipe B: {'Gustavo', 'Fernando'}
```

Diferença Simétrica (^ ou `.symmetric_difference()`): Retorna os elementos que estão em um dos conjuntos, mas **não em ambos**. É o oposto da interseção.

```
1 # Usando o operador ^
2 devs_ao_comuns = devs_a ^ devs_b
3 print(f"Diferença Simétrica: {devs_ao_comuns}")
4 # Saída: Diferença Simétrica: {'Gustavo', 'Fernando', 'Caio', 'Ana'}
```

4.3. Ordem dos conjuntos

Se observar as saídas acima, a ordem de entrada não permanece constante nos conjuntos , portanto vem a pergunta:

Como podemos ordenar conjuntos?

A resposta direta é: **você não consegue manter a ordem dos elementos *dentro* de um conjunto**. Por definição, a estrutura de dados `set` em Python é uma coleção **não ordenada**. A principal vantagem dela é a performance em operações como verificar se um elemento existe, e nas operações matemáticas (união, interseção, etc.), e para isso ela abdica de manter uma ordem.

Como Contornar Isso?

Se a ordem dos elementos é importante para o seu resultado final, o truque é usar os conjuntos como uma ferramenta para descobrir *quais* elementos devem estar no resultado, e depois usar uma lista para manter a ordem.

Veja como aplicar isso ao exemplo que você selecionou:

1. Comece com suas listas originais, que são ordenadas.
2. Use os conjuntos para calcular a operação de forma eficiente.
3. Crie uma nova lista comparando com sua lista original e mantendo apenas os itens que estão no conjunto resultante.

```
1 # O resultado da operação é um conjunto (sem ordem garantida)
2 devs_apenas_no_set_a = set_a - set_b
3 # Resultado: {'Caio', 'Ana'}
4
5 # Agora, para manter a ordem original da primeira lista, nós a filtramos:
6 resultado_ordenado = []
7 # de uma lista vazia. adicionamos apenas os elementos que estão no conjunto resultante
8 for dev in devs_a:
9     if dev in devs_apenas_no_set_a:
10         resultado_ordenado.append(dev)
11
12 print(f"Diferença mantendo a ordem original: {resultado_ordenado}")
13 # Saída: Diferença mantendo a ordem original: ['Ana', 'Caio']
```

5. Dicionários (dict)

Diferente de listas e tuplas, os dicionários não armazenam elementos em uma sequência ordenada, mas sim como um conjunto de pares **chave-valor**. Pense neles como uma agenda de contatos, onde o nome é a "chave" e o telefone é o "valor".

- **Sintaxe:** Pares **chave: valor** separados por vírgula, dentro de chaves `{}`.
- **Características Principais:**
 - **Mutável:** Você pode adicionar, alterar e remover pares chave-valor.
 - **Ordenado (a partir do Python 3.7):** As versões modernas do Python mantêm a ordem de inserção das chaves.
 - **Acessado por Chave:** Os valores são acessados através de suas chaves únicas, não por um índice numérico.
 - **Chaves Únicas:** Não pode haver chaves duplicadas em um dicionário.

```
1 # Criando um dicionário para um aluno
2 aluno = {
3     "nome": "Carlos",
4     "idade": 22,
5     "curso": "Engenharia"
6 }
7
8 # Acessando um valor pela sua chave
9 print(f"O nome do aluno é: {aluno['nome']}")
10
11 # Adicionando um novo par chave-valor
12 aluno["cidade"] = "Rio de Janeiro"
13 print(f"Dicionário atualizado: {aluno}")
```

5.1. Métodos de dicionário

.get(chave, valor_padrao): Retorna o valor para a chave especificada. A grande vantagem é que, se a chave não existir, ele retorna **None** (ou um **valor_padrao** que você definir), **evitando um erro**.

```
1 print(aluno.get("idade"))      # Saída: 23
2 print(aluno.get("apelido"))    # Saída: None
3 print(aluno.get("apelido", "Não informado"))
# Saída: Não informado
```

.update(outro_dicionario): Atualiza o dicionário com os pares chave-valor de outro dicionário. Se a chave já existir, o valor é atualizado.

```
1 aluno = {"nome": "Julia", "idade": 22}
2 aluno.update({"cidade": "Dourados", "curso": "Letras"})
3 # aluno agora é {'nome': 'Julia', 'idade': 23, 'cidade': 'Dourados', 'curso': 'Letras'}
```

.pop(chave, valor_padrao): Remove a chave especificada e **retorna o seu valor**. Gera um erro se a chave não for encontrada, a menos que um **valor_padrao** seja fornecido.

```
1 # aluno = {'nome': 'Julia', 'idade': 23, 'cidade': 'Dourados', 'curso': 'Letras'}
2 curso_removido = aluno.pop("curso")
3 # curso_removido é 'Letras'
4 # aluno agora é {'nome': 'Julia', 'idade': 23, 'cidade': 'Dourados'}
```

.popitem(): Remove e retorna o **último** par chave-valor inserido (como uma tupla). Útil para processar itens em ordem inversa.

```
1 # aluno = {'nome': 'Julia', 'idade': 23, 'cidade': 'Dourados'}
2 ultimo_item = aluno.popitem()
3 # ultimo_item é ('cidade', 'Porto') e foi removido do dicionário
```

.clear(): Remove todos os itens do dicionário, deixando-o vazio.

.keys(): Retorna uma visualização de todas as chaves.

```
1 chaves = aluno.keys()
2 # chaves é dict_keys(['nome', 'idade'])
3 for k in chaves:
4     print(k)
```

.values(): Retorna uma visualização de todos os valores.

```
1 valores = aluno.values()
2 # valores é dict_values(['Julia', 23])
3 for v in valores:
4     print(v)
```

O método **.keys()** e **.values()** retorna um objeto especial chamado **dict_keys** e **dict_values**. Pense nele como uma "visualização" ao vivo dos valores do seu dicionário.

A principal característica é que, se você alterar um valor no dicionário, a visualização (**dict_keys** e **dict_values**) reflete essa mudança automaticamente.

No entanto, para a maioria dos usos práticos, você pode tratá-lo como se fosse uma lista e pode convertê-lo para uma caso precise.

6. Conclusão

Sequências ordenadas são ferramentas úteis e poderosas em python, mas é importante saber qual usar e qual se encaixa melhor com cada momento.

Além de que em python é possível mesclar e utilizar tipos de estruturas conjuntas, como por exemplo:

- Listas de dicionários
- Listas com subconjuntos
- Dicionário com chaves multivalorados (listas)

A dica mais importante para trabalhar com python é **experimente**. se você acha que algo é possível, então provavelmente terá uma maneira de implementar, além de que sempre terão métodos e funções em python para descobrir.